SpendWise

A personal expense tracker, shipped through a full DevSecOps pipeline, from **git push** to a monitored Kubernetes deployment.

GitHub Actions | Trivy | GHCR | Terraform | Kubernetes | Helm | ArgoCD | Prometheus | Grafana

Project overview

Problem: people lose track of small daily spending.

App: log expenses by category, edit or delete them, and see totals per category.

Goal: take that app from *git push* to a monitored, autoscaled Kubernetes deployment with no manual steps.

7
REST endpoints

10
pytest cases

0
root containers

5
Grafana panels

Tech stack

1 App

React 18 + Vite · FastAPI · SQLAlchemy · PostgreSQL 16 · Alembic

2 Quality

pytest + TestClient on an isolated SQLite DB · Pydantic validation

3 Containers

Multi-stage Docker builds · nginx-unprivileged · Docker Compose

4 CI/CD + Security

GitHub Actions · Trivy gate · GHCR with commit-SHA tags

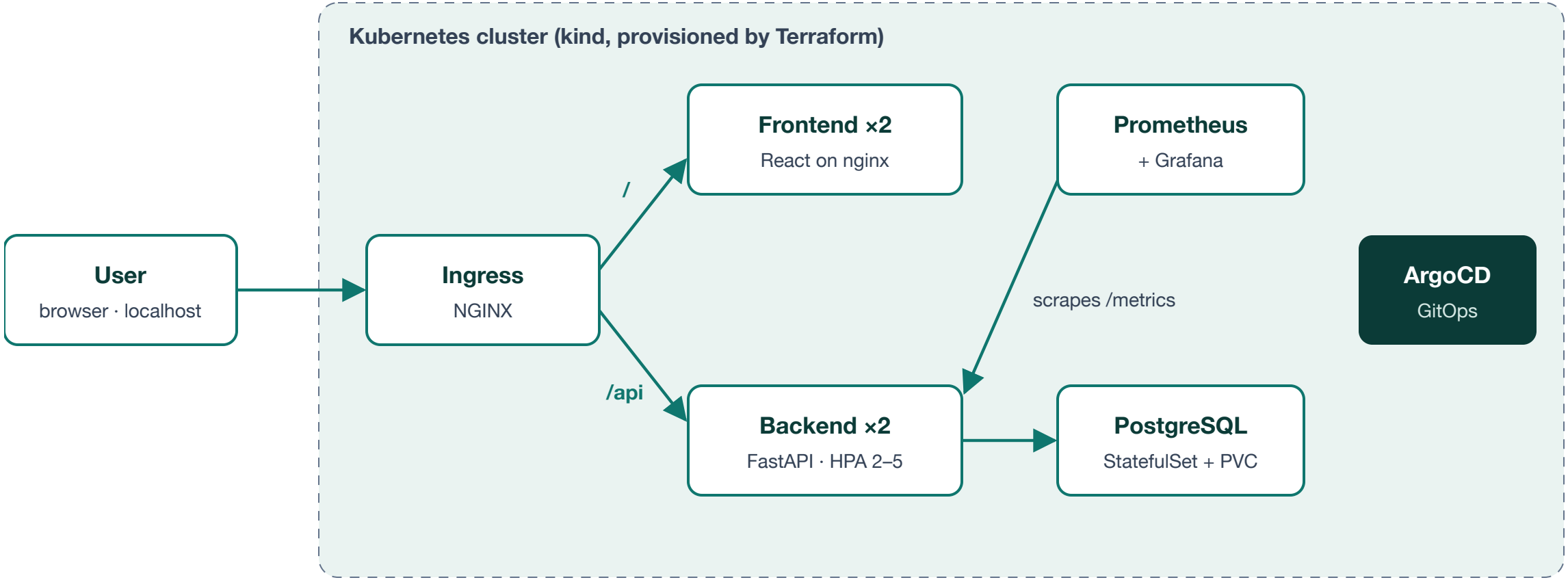
5 Infrastructure

Terraform · kind (Kubernetes 1.35) · AWS VPC + EKS code

6 Deploy + Observe

Helm · ArgoCD · NGINX Ingress · Prometheus · Grafana

Architecture



Images come from GHCR, tagged with the commit SHA. ArgoCD syncs the Helm chart from Git and Prometheus scrapes the backend.

CI/CD pipeline: GitHub Actions

1

Test

pytest -v and the frontend build. Any failure stops the pipeline.

2

Build

docker build of the backend and frontend in a parallel matrix.

3

Scan

Trivy: any fixable HIGH or CRITICAL CVE fails the build.

4

Push

To GHCR as **:<commit-sha>**, never :latest.

5

GitOps

A bot commits the new tag to values.yaml and ArgoCD syncs it.

Runs on every push to main and every pull request. Pull requests build and scan but never push or deploy.

DevSecOps: security built in

Trivy gate

Scans the OS packages and app dependencies of both images. Any fixable HIGH or CRITICAL CVE fails the pipeline, so only clean images reach GHCR.

First run: 7 HIGH CVEs in the backend, e.g. CVE-2025-62727, a Starlette denial-of-service bug. Fixed by upgrading and using a multi-stage build. Now 0 HIGH/CRITICAL.

- ✓ **Non-root containers**
  UID 10001 / 101, all capabilities dropped, seccomp RuntimeDefault
- ✓ **No secrets in Git**
  GITHUB_TOKEN for GHCR; .env, tfvars and tfstate are git-ignored
- ✓ **Least privilege**
  Per-job workflow permissions; credentials in Kubernetes Secrets
- ✓ **Immutable tags**
  Every pod traces back to the commit that built it

Infrastructure as code: Terraform

terraform/local: applied

- kind cluster (Kubernetes 1.35)
- ingress-nginx
- metrics-server (feeds the HPA)
- kube-prometheus-stack
- ArgoCD

init → plan (5 to add) → apply → destroy

terraform/: AWS target

- VPC 10.0.0.0/16 across 2 AZs
- 2 public + 2 private subnets, NAT gateway
- EKS + managed node group (2 × t3.medium)
- EBS CSI driver via IRSA

Written and validated. Apply is blocked until the AWS account activates.

Kubernetes: Helm + ArgoCD

Deployments

Backend and frontend with 2 replicas each, HPA 2→5 on CPU, rolling updates, probes

Services

ClusterIP for the backend :8000, frontend :80 and postgres :5432

Ingress

NGINX routes / to the frontend and /api to the backend

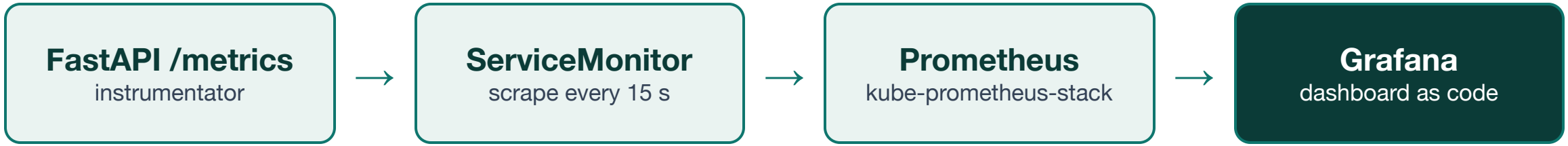
Data

Postgres StatefulSet on a persistent volume; credentials in a Secret

GitOps loop

1. CI pushes the image :sha
2. CI commits the tag to values.yaml
3. ArgoCD detects the change
4. Helm renders and does a rolling update
5. selfHeal reverts manual changes

Observability: Prometheus + Grafana



Request rate
req/s per endpoint

p95 latency
histogram_quantile

Error rate
5xx as % of traffic

Status mix
2xx / 4xx / 5xx

The dashboard ships as a ConfigMap and Grafana's sidecar loads it automatically. scripts/load-test.sh generates traffic for the demo.

Key learnings and next steps

- Shift left: tests and Trivy run before anything is pushed or deployed
- Git is the single source of truth: SHA tags and ArgoCD make deploys auditable and reversible
- Non-root and least-privilege defaults cost little when set up from day one
- Terraform can manage local platforms too: one apply builds the whole cluster stack
- Keep a local fallback: cloud accounts can be blocked by activation or billing

Next steps

- Apply the AWS Terraform (VPC + EKS)
- User auth (OAuth2 + JWT)
- TLS with cert-manager
- Managed database (Amazon RDS)
- External Secrets